Wavelet Convolutions Are Memory-Bound: An I/O-Aware Formulation of WTConv

Anonymous authors
Paper under double-blind review

Abstract

Wavelet convolution (WTConv) expands a network's receptive field exponentially with the number of decomposition levels while keeping parameter count linear, providing an efficient drop-in replacement for standard convolutions. However, because its reference implementation operates at an arithmetic intensity of 0.4–0.9 FLOP/byte, the operator is severely memory-bound. When configured with 3×3 kernels to replace large dense convolutions (such as 7×7), the reference implementation is slower in wall-clock time due to excessive data movement through high-bandwidth memory (HBM). We resolve this bottleneck with an exact, I/O-aware formulation based on three algebraic reformulations: (1) recomputing the multiplication-free Haar analysis in registers on chip, (2) collapsing the multi-level synthesis cascade into a single closed-form pass indexed by output coordinate bits, and (3) folding learned per-channel scales into the convolution weights. The resulting formulation is a bit-exact reassociation of the operator that reduces HBM traffic by 2.5–2.6× across all levels (moving under $8.3N$ elements for any level count L, compared to up to $21.3N$ for the reference). A CUDA implementation is 4.48–5.32× faster than the reference in `fp32` and 4.40–4.84× in `fp16` while needing 1.85–2.34× less memory, making WTConv 2.57–3.08× faster than the dense 7×7 convolution it is proposed to replace, at every level.

1 Introduction

Convolutional networks acquire large receptive fields slowly. Stacking 3×3 kernels grows the theoretical receptive field linearly in depth and the *effective* receptive field only as $\mathcal{O}(\sqrt{n})$ (Luo et al., 2016), which is a large part of why vision transformers, whose first layer already mixes globally (Dosovitskiy et al., 2021; Liu et al., 2021), were initially so competitive. The convolutional response has been to enlarge kernels directly: 31×31 re-parameterised depthwise kernels (Ding et al., 2022), or 51×51 sparse factorised ones (Liu et al., 2023). Both buy receptive field at a parameter and FLOP cost that grows with the kernel area.

WTConv (Finder et al., 2024) takes a different route. It applies a small $k \times k$ depthwise kernel independently at each level of an L-level Haar decomposition. Because level ℓ operates on a 2^ℓ-times-downsampled grid, its footprint in input pixels is $k \cdot 2^\ell$, so an L-level layer reaches $k \cdot 2^L$ pixels using $\mathcal{O}(Lk^2C)$ parameters: the receptive field grows exponentially in L while parameters grow linearly, i.e. parameters grow only logarithmically in the receptive field. It is a drop-in replacement for depthwise convolutions and improves accuracy and robustness in ConvNeXt and MobileNetV2 backbones (Liu et al., 2022; Sandler et al., 2018).

There is a catch that the asymptotics hide. In the reference implementation, a WTConv layer is *slower than the dense* 7×7 *convolution it is supposed to replace*, by 1.46–2.07× in our measurements. An operator that is cheaper in parameters and FLOPs but more expensive in wall-clock time is a poor trade in practice, and this is the gap we close.

The cost of WTConv is data movement, not arithmetic. Our starting observation is that WTConv is nowhere near compute-bound. Counting the multiply–accumulates the operator actually performs against the bytes its reference implementation actually moves gives an arithmetic intensity of 0.44–0.89 FLOP/byte (Section 3). An NVIDIA RTX A6000 balances at roughly 50 FLOP/byte. The operator therefore sits two

orders of magnitude to the left of the roofline ridge: the GPU spends essentially all of its time waiting for memory, and the arithmetic, including the Haar transform itself, is free in comparison. This reframes the problem. The question is not how to compute the wavelet transform faster; it is how to stop writing intermediate tensors to high-bandwidth memory (HBM). The same reframing drove FlashAttention (Dao et al., 2022), and the analogy is close: in both cases a mathematically elegant operator was expressed as a chain of memory-bound primitives, and in both cases the fix is a reassociation that keeps intermediates on chip.

What makes WTConv especially amenable. Two structural properties of the Haar basis make the reassociation unusually clean, and they are what we exploit.

Haar analysis has low arithmetic cost. The 2×2 transform uses only sums, differences, and fixed scaling by $\frac{1}{2}$. Recomputing its coefficients is therefore inexpensive compared with storing and reloading them. Longer wavelet filters would make this trade-off less favourable.

The normalised Haar operator is symmetric and self-inverse. With the $\frac{1}{2}$ normalisation, the 4×4 analysis matrix satisfies $\mathbf{H} = \mathbf{H}^\top = \mathbf{H}^{-1}$ exactly (Proposition 1). Consequently, analysis and synthesis exchange roles during backpropagation, and no separate derivative of the Haar transform is needed.

Contributions.

- **A cost model for WTConv** (Section 3). We give an exact element-by-element I/O accounting of the reference implementation: $Q_{\text{ref}} = 7N + \frac{43}{3}N(1 - 4^{-L})$. We place the operator on the roofline and show it is bandwidth-bound by two orders of magnitude.
- **An I/O-reduced reformulation** (Section 4) built from three exact algebraic identities (register-resident analysis, closed-form single-pass synthesis across all levels, and scale folding), reaching $7N$ at L=1 and at most $\approx 8.3N$ for any L, against $2N$ of compulsory traffic — a 2.5–2.6× reduction in traffic across all levels. The synthesis collapse (Section 4.2) turns an L-level inverse cascade into a single closed-form pass whose per-level sign pattern is read off the bits of the output coordinate, eliminating all intermediate low-pass planes and sequential level dependencies uniformly across all levels.
- **Measurement** (Section 5). Over 864 measured configurations, a CUDA realisation is 4.48–5.32× faster than the reference in `fp32` and 4.40–4.84× in `fp16`, needs 1.85–2.34× less memory, and is faster than the dense 7×7 convolution WTConv is proposed to replace at every level. We report traffic and latency separately and do not claim a proportionality between them (Sections 4.4 and 5.4).
- **Exactness, not approximation.** Every step is a reassociation of the same multilinear map, so agreement with the reference is limited only by floating-point reassociation error, which we quantify for the forward pass and for every gradient the layer produces.

We present and evaluate a custom CUDA implementation of this formulation.

2 Background

2.1 The WTConv operator

Let $\mathsf{X} \in \mathbb{R}^{B \times C \times H \times W}$ denote the input, and let $N = B \cdot C \cdot H \cdot W$ be its element count, the unit in which we quote every I/O cost. Let $*_{\text{dw}}$ denote depthwise convolution (one $k \times k$ kernel per channel, zero-padded to preserve resolution) and let $\odot$ denote per-channel scaling.

Let $\mathcal{A}$ denote one level of Haar analysis, which splits each channel into four half-resolution subbands $(\text{LL}, \text{LH}, \text{HL}, \text{HH})$, and let $\mathcal{A}^\top$ denote its synthesis, which inverts it exactly (Proposition 1). For a tensor carrying a subband axis, let $[\,\cdot\,]_{\text{LL}}$ denote the low-pass band, $[\,\cdot\,]_{\text{H}}$ the three high-pass bands, and $\|$ their concatenation. WTConv preserves the channel count. It learns a $k \times k$ kernel and a per-channel scale at

each level, $(\mathbf{W}^{(\ell)}, \mathbf{s}^{(\ell)})$ for $\ell = 1, \ldots, L$, together with a full-resolution base convolution $(\mathbf{W}^{(0)}, b, \mathbf{s}^{(0)})$, giving $\mathcal{O}(Lk^2C)$ parameters in total. Algorithm 1 states the operator as the reference implements it.

Algorithm 1 WTConv, reference form (Finder et al., 2024)

Require: input X, levels L, parameters $\{\mathbf{W}^{(\ell)}, \mathbf{s}^{(\ell)}\}_{\ell=0}^{L}$, bias b

$\mathsf{X}^{(0)} \leftarrow \mathsf{X}$ ▷ decomposition carrier
for $\ell = 1, \ldots, L$ **do** ▷ downward: decompose and filter
  $\mathsf{Y}^{(\ell)} \leftarrow \mathcal{A}\,\mathsf{X}^{(\ell-1)}$ ▷ Haar analysis
  $\mathsf{Z}^{(\ell)} \leftarrow \mathbf{s}^{(\ell)} \odot \left(\mathsf{Y}^{(\ell)} *_{\mathrm{dw}} \mathbf{W}^{(\ell)}\right)$ ▷ filter, then scale
  $\mathsf{X}^{(\ell)} \leftarrow \left[\mathsf{Y}^{(\ell)}\right]_{\mathrm{LL}}$ ▷ carry the *raw*, unfiltered low-pass band
end for
$R^{(L+1)} \leftarrow 0$
for $\ell = L, \ldots, 1$ **do** ▷ upward: reconstruct and accumulate
  $R^{(\ell)} \leftarrow \mathcal{A}^{\top}\left(\left(\left[\mathsf{Z}^{(\ell)}\right]_{\mathrm{LL}} + R^{(\ell+1)}\right) \,\middle\|\, \left[\mathsf{Z}^{(\ell)}\right]_{\mathrm{H}}\right)$
end for
return $\mathbf{s}^{(0)} \odot \left(\mathsf{X} *_{\mathrm{dw}} \mathbf{W}^{(0)} + b\right) + R^{(1)}$ ▷ base path + reconstruction

2.2 Haar transform differentiation

For each non-overlapping 2×2 block, the normalised Haar transform maps four input samples to four subband coefficients using the following matrix.

Proposition 1 (Symmetry and self-inversion)**.** $\mathbf{H} = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$ *satisfies* $\mathbf{H}^{\top} = \mathbf{H}$ *and* $\mathbf{H}^2 = \mathbf{I}$*. Therefore,* $\mathbf{H}^{-1} = \mathbf{H}^{\top} = \mathbf{H}$.

The full analysis operator $\mathcal{A}$ applies $\mathbf{H}$ independently to every block, and synthesis applies $\mathcal{A}^{\top}$. For $\mathbf{y} = \mathbf{H}\mathbf{x}$, the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{H}^{\top}\frac{\partial \mathcal{L}}{\partial \mathbf{y}} = \mathbf{H}\frac{\partial \mathcal{L}}{\partial \mathbf{y}}. \tag{1}$$

Thus, backward through analysis applies synthesis, and backward through synthesis applies analysis. The Haar step needs no separate derivative; convolution and scale gradients are computed as usual.

2.3 The roofline

The roofline model (Williams et al., 2009) bounds attainable throughput by $\min(\pi, \beta I)$ where π is peak compute, β peak bandwidth, and I the arithmetic intensity in FLOP/byte. The ridge point $I^{\star} = \pi/\beta$ separates memory-bound from compute-bound operation. For the RTX A6000 used throughout, $\beta = 768$ GB/s and $\pi \approx 38.7$ TFLOP/s in `fp32`, so $I^{\star} \approx 50$ FLOP/byte. Any operator with $I \ll I^{\star}$ is bandwidth-limited: its runtime is governed by the bytes it moves rather than by how the arithmetic is scheduled. The bound is an upper bound on attainable throughput and not an equality, so this identifies traffic as the quantity to minimise without implying that runtime is proportional to it. The practical content of this paper is that WTConv is deeply in that regime, so minimising traffic is not merely one optimisation among many: it is the whole problem.

3 WTConv is memory-bound

3.1 I/O cost of the reference implementation

The reference implementation expresses each level as a chain of framework primitives, every one of which reads its input from HBM and writes its output back (Figure 1a). We count elements crossing the HBM boundary: each stage reads its input tensor once and writes its output tensor once, weight traffic is $\mathcal{O}(Ck^2)$ and therefore negligible against $\mathcal{O}(N)$. Writing $N_\ell = N/4^{\ell-1}$ for the element count entering level ℓ:

- *Analysis*, per level: the Haar transform is executed as a grouped stride-2 `conv2d` against a constant $\pm\frac{1}{2}$ filter ($2N_\ell$); the depthwise convolution over the $4C$-channel coefficient tensor ($2N_\ell$); the scale multiply ($2N_\ell$). Total $6N_\ell$.
- *Synthesis*, per level: the cross-level low-pass add ($\frac{3}{4}N_\ell$); the concatenation of the summed low-pass band with the three high bands ($2N_\ell$); the `conv_transpose2d` ($2N_\ell$). Total $\frac{19}{4}N_\ell$.
- *Base path*: convolution ($2N$), scale multiply ($2N$), final add ($3N$). Total $7N$.

Using $\sum_{\ell=1}^{L} 4^{-(\ell-1)} = \frac{4}{3}(1 - 4^{-L})$,

$$Q_{\text{ref}} \;=\; 7N \;+\; \Big(6 + \tfrac{19}{4}\Big) \cdot \tfrac{4}{3}\, N\big(1 - 4^{-L}\big) \;=\; 7N + \tfrac{43}{3}\, N\big(1 - 4^{-L}\big), \tag{2}$$

rising from $17.75N$ at $L = 1$ to $21.33N$ as $L \to \infty$; that is, evaluating the layer moves between 18 and 21 times the input tensor's worth of data through HBM. For a representative B=8, C=64, H=W=256 input, $N = 33.6$M elements (128 MiB in `fp32`) and the reference therefore moves roughly 2.9 GB per forward pass.

3.2 Arithmetic intensity of the reference implementation

The base convolution performs k^2 multiply–accumulates per element. At level ℓ, the depthwise convolution over the coefficient tensor costs a further k^2 per element of N_ℓ, and the analysis and synthesis each cost 4, since the reference executes both as 2×2 convolutions rather than as additions. Summing over levels with the same geometric factor as before,

$$\text{MAC} \;=\; N\big[k^2 + (k^2 + 8) \cdot \tfrac{4}{3}\big(1 - 4^{-L}\big)\big]. \tag{3}$$

For $k = 3$ and $L = 5$ this is $31.6N$ multiply–accumulates, or $63.3N$ FLOPs, against $4Q_{\text{ref}} = 85.3N$ bytes in `fp32`:

$$I_{\text{ref}} \;=\; \frac{63.3N}{85.3N} \;\approx\; 0.74 \text{ FLOP/byte}. \tag{4}$$

Against the ridge point $I^\star \approx 50$ (subsection 2.3), WTConv therefore sits almost 70× to the left of the ridge, using under 2% of the machine's arithmetic capability. The arithmetic is not the problem: no amount of faster transform evaluation would help, and only removing traffic can.

4 An I/O-Aware formulation

To eliminate the severe memory bottleneck of the reference WTConv (Equation (2)), we derive an I/O-aware formulation based on three exact algebraic reformulations. These reformulations leave the underlying mathematical operator unchanged, differing from the reference implementation solely in floating-point evaluation order. Together, they eliminate all large intermediate subband tensors from high-bandwidth memory (HBM) by performing multi-stage computations directly in GPU registers and shared memory across all decomposition levels.

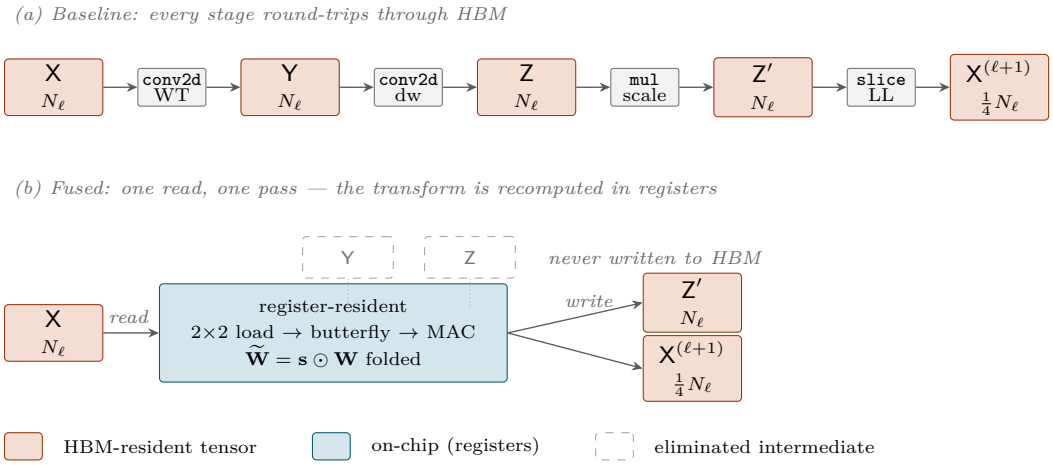

Figure 1: HBM dataflow for one WTConv decomposition level ℓ, with $N_\ell = N/4^{\ell-1}$ the number of elements entering that level. (a) The reference implementation expresses the level as a chain of framework primitives; every stage materialises a full-size tensor in HBM. (b) The fused formulation reads the input once, evaluates the Haar butterfly and the depthwise convolution on chip against weights that already carry the learned scale, and writes only the results. Y and Z are never written,

4.1 Fusing Haar analysis with depthwise convolution

For a 2×2 spatial patch $\left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$, the normalized 2D Haar analysis transform maps four input pixels to subband coefficients $(\mathrm{LL}, \mathrm{LH}, \mathrm{HL}, \mathrm{HH})$ via:

$$\begin{aligned} \mathrm{LL} &= \tfrac{1}{2}(a+b+c+d), & \mathrm{LH} &= \tfrac{1}{2}(a+b-c-d), \\ \mathrm{HL} &= \tfrac{1}{2}(a-b+c-d), & \mathrm{HH} &= \tfrac{1}{2}(a-b-c+d). \end{aligned} \tag{5}$$

Evaluating Equation (5) requires only additions and scaling by $\frac{1}{2}$, with zero floating-point multiplications. In contrast, the reference implementation materializes the entire coefficient tensor $\mathsf{Y}^{(\ell)}$ in HBM via grouped convolutions with $\pm\frac{1}{2}$ filters. Each level runs Haar analysis, depthwise convolution, and learned scaling as separate passes. Each pass reads and writes an N_ℓ-element tensor, moving $6N_\ell$ elements in total. Our fused approach computes the Haar coefficients inside the depthwise convolution and folds the scale into its weights (Section 4.3). Because WTConv reaches only 0.74 FLOP/byte (Section 3), recomputing the coefficients is much cheaper than materialising them in HBM.

Naive fusion would recompute the transform for every convolution tap and load each input pixel k^2 times. Instead, each thread block transforms the 2×2 blocks for one output tile, including a coefficient-space halo of radius $R = (k-1)/2$, and stages the four subbands in shared memory. All taps reuse this copy, so each block is transformed once per tile. Except at the deepest level, the fused approach reads N_ℓ input elements, writes N_ℓ filtered coefficients for synthesis, and writes $\frac{1}{4}N_\ell$ unfiltered low-pass coefficients for the next level. It therefore moves $\frac{9}{4}N_\ell$ elements, versus $6N_\ell$ for the reference. The deepest level has no successor and moves $2N_L$.

4.2 Collapsing the synthesis cascade

The reference implementation executes reconstruction as an L-step sequential loop, where each level reads the lower-level reconstruction from HBM, adds it to its low-pass subband, and applies transposed convolution. This imposes L strict sequential dependencies and incurs an I/O cost of $\frac{19}{4}N_\ell$ per level.

Because Haar synthesis is linear, the low-pass carrier acts as a linear accumulator across levels. As a result, the multi-level synthesis cascade can be collapsed into a single closed-form pass across all L levels. Haar synthesis at level ℓ expands each subband coefficient over a $2^\ell \times 2^\ell$ pixel block using a fixed sign pattern determined by the spatial location of the output pixel within that block.

Proposition 2 (Bit-indexed single-pass synthesis)**.** *Let (y, x) denote an output pixel coordinate, and let level $\ell \in \{1, \dots, L\}$ have subband coefficients $(c_{\mathrm{LL}}^{(\ell)}, c_{\mathrm{LH}}^{(\ell)}, c_{\mathrm{HL}}^{(\ell)}, c_{\mathrm{HH}}^{(\ell)})$ at coarse grid location $(\lfloor y/2^{\ell} \rfloor, \lfloor x/2^{\ell} \rfloor)$. Define the coordinate parity signs at level ℓ as:*

$$s_y^{(\ell)} = (-1)^{\lfloor y/2^{\ell-1} \rfloor \bmod 2}, \qquad s_x^{(\ell)} = (-1)^{\lfloor x/2^{\ell-1} \rfloor \bmod 2}. \tag{6}$$

The reconstructed pixel value $R_{y,x}$ across all L levels is given by:

$$R_{y,x} = \sum_{\ell=1}^{L} 2^{-\ell} \Big(c_{\mathrm{LL}}^{(\ell)} + s_y^{(\ell)} c_{\mathrm{LH}}^{(\ell)} + s_x^{(\ell)} c_{\mathrm{HL}}^{(\ell)} + s_y^{(\ell)} s_x^{(\ell)} c_{\mathrm{HH}}^{(\ell)} \Big). \tag{7}$$

By evaluating Equation (7) in a single pass, a GPU thread computing pixel (y, x) iterates from level L down to 1, addressing coarse coefficients via coordinate bit-shifts and obtaining signs through bitwise parity checks. This eliminates all intermediate low-pass tensors and sequential kernel dependencies. Furthermore, the base-path convolution output is folded directly into the final output store, eliminating a separate full-resolution tensor addition.

4.3 Folding the learned scale

The reference implementation applies learned per-channel scaling s_c after convolution ($\mathsf{Z}_c = s_c(\mathsf{X} *_{\mathrm{dw}} \mathbf{W})_c$) as a standalone elementwise pass. At an arithmetic intensity of $\frac{1}{4}$ FLOP/byte, this pass severely limits memory throughput.

By the linearity of convolution, the scale factor can be folded directly into the depthwise weights:

$$\widetilde{\mathbf{W}}_c = s_c \odot \mathbf{W}_c. \tag{8}$$

This allows $\mathsf{Z} = \mathsf{X} *_{\mathrm{dw}} \widetilde{\mathbf{W}}$ to be computed in a single pass with zero extra runtime cost. The weight folding is performed on host memory with negligible cost $\mathcal{O}(Ck^2)$.

During backpropagation, exact parameter gradients with respect to the unscaled weight $\mathbf{W}_c$ and scale s_c are efficiently recovered via:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_c} = s_c \, \frac{\partial \mathcal{L}}{\partial \widetilde{\mathbf{W}}_c}, \qquad \frac{\partial \mathcal{L}}{\partial s_c} = \left\langle \frac{\partial \mathcal{L}}{\partial \widetilde{\mathbf{W}}_c}, \mathbf{W}_c \right\rangle. \tag{9}$$

4.4 The resulting I/O cost

Collecting the reformulations across all levels $1 \dots L$, with $N_\ell = N/4^{\ell-1}$:

- *Base path*: the folded vendor convolution reads N and writes N ($2N$).
- *Analysis & Depthwise pass*, levels $1 \le \ell \le L$: reads N_ℓ, writes N_ℓ convolved coefficients and $\frac{1}{4}N_\ell$ raw low-pass coefficients ($\frac{9}{4}N_\ell$ each, less the $\frac{1}{4}N_L$ the deepest level does not write).
- *Single-pass Synthesis*: reads all subband coefficients $\sum_{\ell=1}^{L} N_\ell$ and base-path output N, writing the final output N ($\sum_{\ell=1}^{L} N_\ell + 2N$).

Using $\sum_{\ell=1}^{L} N_\ell = \frac{4}{3}N\big(1 - 4^{-L}\big)$, the total memory traffic is:

$$Q_{\mathrm{fused}} \;=\; 4N \;+\; \tfrac{13}{3}\,N\big(1 - 4^{-L}\big) \;-\; \tfrac{1}{4}N\,4^{1-L}. \tag{10}$$

Comparing this to the reference memory traffic Q_{ref} (Equation (2)), the theoretical reduction ratio in HBM data movement is:

$$\frac{Q_{\mathrm{ref}}}{Q_{\mathrm{fused}}} = \frac{7 + \frac{43}{3}(1 - 4^{-L})}{4 + \frac{13}{3}(1 - 4^{-L})} \xrightarrow[L \to \infty]{} \frac{64}{25} = 2.56. \tag{11}$$

As $L \to \infty$, the fused formulation achieves up to a 2.56× reduction in memory traffic over the reference implementation. Table 1 summarizes the exact element counts and traffic reduction ratios for decomposition levels $L = 1 \dots 5$.

Table 1: Elements crossing the HBM boundary per forward evaluation, in units of $N = B \cdot C \cdot H \cdot W$, from Equation (2), Equation (10), and Equation (11). The last row is a ratio of *traffic*, not of runtime; see the discussion below.

	Decomposition levels L				
	1	2	3	4	5
Reference, Q_{ref}	$17.75N$	$20.44N$	$21.11N$	$21.28N$	$21.32N$
Fused, Q_{fused}	$7.00N$	$7.94N$	$8.19N$	$8.25N$	$8.27N$
Traffic removed	2.54×	2.57×	2.58×	2.58×	2.58×

5 Results

5.1 Setup

All measurements are single-layer microbenchmarks on NVIDIA RTX A6000 (sm_86, 47.5 GiB), PyTorch 2.6.0+cu124, CUDA 12.4, cuDNN 90100, Triton 3.2.0, in eager mode on both arms. We sweep $C \in \{32, 64, 128\}$ and $H = W \in \{128, 256, 512\}$ at B=8 and $L \in \{1, \ldots, 5\}$, in `fp32` and `fp16`, over both protocols below and over both a forward pass and a full training step: 864 measurements in all. Each configuration runs 20 warmup iterations followed by 50 iterations timed individually with CUDA events, so device time is measured rather than host-side launch time. Peak memory is the allocator high-water mark (`torch.cuda.max_memory_allocated`) over one untimed step. The full protocol, including the pinned tolerances and the weight synchronisation between the two arms, is in Section A.

Baselines. WTConv always runs at k=3, in both the reference form of Algorithm 1 and the fused form of Section 4. The plain convolutions appear twice, under two protocols that answer two different questions. At *matched kernel size* (k=3) they isolate the cost of the wavelet machinery itself: what does an L-level decomposition cost over a single 3×3 convolution? At *matched receptive field* (k=7) they are the comparison a practitioner actually faces, since WTConv is proposed as a replacement for a large convolution and an L-level layer at k=3 already reaches $3 \cdot 2^L \geq 12$ pixels at L=2. Depthwise is `nn.Conv2d(C, C, k, groups=C, padding='same')` and dense is the same without `groups`; the dense baseline is the functionally comparable one, since a depthwise layer does not mix channels.

Reporting convention. Every entry in Tables 3 to 5 is the geometric mean, over the (C, H) sweep, of the per-configuration ratio between the reference WTConv and the method in question; the reference is therefore 1.00× by construction and higher is better throughout. Per-iteration dispersion is small: over all 864 measurements the median p_{10}–p_{90} spread is 0.8% of the mean and the 95th percentile is 4.0%, so the ratios below are not noise-limited. A single configuration reaches 15.7%; the raw JSON carries the full distribution for every measurement and we recommend consulting it before quoting any individual entry. Table 2 gives one configuration in absolute units, since ratios alone hide the scale: at B=8, C=64, H=W=256, L=3 in `fp32`, one forward pass takes 8.98 ms in the reference implementation, 4.65 ms in a dense 7×7 convolution and 1.91 ms in the fused layer.

The two arms compute the same function. Both modules are built from the same seed and every learnable tensor is copied from the reference into the fused layer before measurement, so the comparison is between two evaluations of one function. Maximum absolute deviation over the whole tensor is 2.4×10^{-7} in `fp32`, 2.0×10^{-3} in `fp16` and 1.6×10^{-2} in `bf16`, for the forward output and for ∇_X, at every level $L \in \{1, \ldots, 5\}$; parameter gradients agree to 9.5×10^{-7}–4.8×10^{-6} in `fp32`. These are the residuals of a k^2-term accumulation in the working precision, which is what Section 4 predicts, since each step there is an algebraic identity.

5.2 Latency

Table 3 reports forward latency. Three readings matter.

Table 2: Absolute cost of one layer at B=8, C=64, H=W=256, L=3, `fp32`: mean latency over 50 timed iterations (CUDA events) and allocator high-water mark. Ratios in the other tables are geometric means over the whole sweep; this table fixes one configuration so the scale is visible.

Method	Forward (ms)	Fwd+Bwd (ms)	Peak memory (MiB)
Depthwise conv, k=3	0.655	2.887	576.0
Dense conv, k=3	1.659	5.666	770.3
Depthwise conv, k=7	2.226	11.028	576.0
Dense conv, k=7	4.651	15.763	797.7
WTConv, reference	8.978	30.702	1376.1
WTConv, **fused**	1.911	6.968	752.1

Table 3: Forward latency of every method relative to the reference WTConv (1.00×; higher is faster). WTConv runs at k=3 throughout; the plain convolutions appear both at the same kernel size and at k=7, which is the receptive-field-matched comparison a practitioner faces. Each entry is the geometric mean of per-configuration ratios over $C \in \{32, 64, 128\}$ and H=$W \in \{128, 256, 512\}$ at B=8.

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=3	`fp32`	11.14×	14.17×	15.20×	15.57×	15.87×
	`fp16`	10.54×	12.92×	13.58×	13.82×	13.96×
Dense conv, k=3	`fp32`	4.41×	5.61×	6.02×	6.17×	6.29×
	`fp16`	4.18×	5.13×	5.39×	5.49×	5.54×
Depthwise conv, k=7	`fp32`	3.38×	4.30×	4.61×	4.72×	4.81×
	`fp16`	1.98×	2.42×	2.55×	2.59×	2.62×
Dense conv, k=7	`fp32`	1.46×	1.85×	1.99×	2.03×	2.07×
	`fp16`	1.60×	1.96×	2.06×	2.09×	2.12×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	4.48×	5.06×	5.24×	5.28×	5.32×
	`fp16`	4.40×	4.81×	4.84×	4.82×	4.77×

The reference operator loses to the convolution it replaces. At matched receptive field, a dense 7×7 convolution is 1.46–2.07× faster than the reference WTConv in `fp32` and 1.60–2.12× faster in `fp16`, and a depthwise 7×7 convolution is faster still. WTConv's parameter advantage is real, but in the reference implementation it is paid for in wall-clock time, which is the gap this paper set out to close.

Fusion closes it. The fused layer is 4.48–5.32× faster than the reference in `fp32` and 4.40–4.84× in `fp16`. The ratio grows with L, monotonically in `fp32` and up to L=3 in `fp16`, because deeper decompositions give the reference more intermediates to materialise while Equation (10) is nearly flat in L. Against the dense 7×7 convolution it is now 2.57–3.08× faster (`fp32`; 2.25–2.76× in `fp16`), and it also beats the much cheaper depthwise 7×7 baseline by 1.11–1.33× in `fp32` and 1.82–2.23× in `fp16`. The verdict of the previous paragraph is reversed at every level and in both precisions.

What remains. At matched kernel size the fused layer sits at parity with a single dense 3×3 convolution (`fp32`: 1.02× at L=1, 0.85× at L=5), while spanning 96 pixels at L=5 rather than three, and remains 2.4–3.0× slower than a single depthwise 3×3 convolution. That residual gap is the honest statement of what the operator now costs: an L-level decomposition still writes and re-reads $\frac{4}{3}N(1 - 4^{-L})$ coefficient elements that a single depthwise convolution does not, as Equation (10) makes explicit. Notably the fused speedup is slightly *smaller* in `fp16` than in `fp32`: half precision halves the bytes moved by both arms, and the reference, which moves more of them, benefits more.

Table 4: Latency of a full training step (forward and backward) relative to the reference WTConv (1.00×; higher is faster), same sweep and same conventions as Table 3. The backward pass moves the same intermediates in the opposite direction, so the reformulation applies to it unchanged.

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=3	`fp32`	7.51×	9.92×	10.57×	10.83×	10.98×
	`fp16`	9.71×	12.81×	13.73×	14.11×	14.38×
Dense conv, k=3	`fp32`	3.88×	5.13×	5.47×	5.60×	5.68×
	`fp16`	3.76×	4.96×	5.31×	5.46×	5.56×
Depthwise conv, k=7	`fp32`	1.99×	2.63×	2.80×	2.87×	2.91×
	`fp16`	1.37×	1.81×	1.94×	2.00×	2.04×
Dense conv, k=7	`fp32`	1.15×	1.52×	1.62×	1.66×	1.69×
	`fp16`	1.39×	1.83×	1.96×	2.01×	2.05×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	3.86×	4.50×	4.63×	4.67×	4.68×
	`fp16`	3.31×	3.82×	3.93×	3.94×	3.94×

Table 5: Peak allocated memory over a training step relative to the reference WTConv (1.00×; higher means the method needs less memory), same sweep as Table 3. Measured as the CUDA allocator high-water mark, not reserved memory. Fusion removes the per-level coefficient tensors from the autograd tape, which is where the saving comes from.

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=3	`fp32`	1.59×	2.32×	2.32×	2.32×	2.32×
	`fp16`	1.38×	2.01×	2.01×	2.01×	2.01×
Dense conv, k=3	`fp32`	1.60×	2.33×	2.33×	2.33×	2.33×
	`fp16`	1.52×	2.22×	2.22×	2.22×	2.22×
Depthwise conv, k=7	`fp32`	1.59×	2.32×	2.32×	2.32×	2.32×
	`fp16`	1.38×	2.01×	2.01×	2.01×	2.01×
Dense conv, k=7	`fp32`	1.51×	2.19×	2.19×	2.19×	2.19×
	`fp16`	1.48×	2.15×	2.15×	2.15×	2.15×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	1.85×	2.34×	2.29×	2.28×	2.28×
	`fp16`	1.85×	2.34×	2.29×	2.28×	2.27×

Training step. Table 4 repeats the measurement over a forward and backward pass. The reformulation carries over: the fused layer is 3.86–4.68× faster than the reference in `fp32` and 3.31–3.94× in `fp16`, and 2.77–3.34× faster than the dense 7×7 convolution. The ratios are somewhat lower than for the forward pass alone, and the reason is that a training step contains work fusion does not touch: both arms obtain the base-path gradients from the same vendor primitive, and both must run a per-level weight-gradient reduction, so the fraction of the step that the reformulation addresses is smaller than in the forward pass. What the reformulation does supply for free is the backward Haar step itself, which by Proposition 1 is the forward transform again and needs no separate kernel.

5.3 Peak memory

Table 5 reports the allocator high-water mark over a training step. The reference implementation needs 1.85–2.34× the memory of the fused layer, and the reduction is essentially flat in L beyond L=2, because the dominant saved allocation is the level-1 coefficient tensor: it holds N elements, as many as the input and three times as many as all deeper levels combined ($\sum_{\ell \geq 2} N_\ell \to N/3$). The mechanism is the one Section 4 describes. Every per-level subband tensor the reference writes to HBM is also retained by autograd until the backward pass, whereas the fused layer recomputes the Haar coefficients on chip and keeps only the filtered coefficients it must.

The comparison against the plain convolutions is the more interesting one. At matched receptive field the fused layer's training-step footprint is 0.82–0.96× that of the dense 7×7 convolution, i.e. at or below it, and 0.86–1.02× that of a depthwise 3×3 convolution, i.e. within a few percent of every plain-convolution baseline in the table. For inference (Table 6 in Section B, no autograd tape) the fused layer needs 1.09–1.27× the memory of the dense 7×7 convolution and 1.58–1.84× that of a depthwise 3×3 one; the wavelet path still holds L coefficient buffers, which no plain convolution needs.

5.4 Traffic is a lower bound on the gain, not a predictor of it

Section 4.4 predicts a 2.54–2.58× reduction in HBM traffic; the measured forward speedup is 4.48–5.32×. The measurement *exceeds* the traffic model, and it is worth being precise about why, since the naive reading, that we removed more bytes than we claimed, is wrong.

Equation (2) counts compulsory tensor reads and writes and nothing else. It does not count kernel launches, of which the reference issues $6L + 3$ (six stages per level, three on the base path) against $L + 2$ for the fused layer, whose only additional work is the $\mathcal{O}(Ck^2)$ weight fold, and it assumes both arms convert bytes into time at the same rate, which they do not: the reference's deepest levels operate on tensors $4^{-(L-1)}$ the size of the input, too small to saturate the machine, so six of its kernels per level run at a fraction of peak bandwidth, whereas the fused layer visits every level's coefficients from a single full-resolution synthesis pass. Here the traffic account therefore identifies the quantity to remove and bounds the improvement from below rather than estimating it; the remaining factor comes from launch overhead and from the occupancy of the passes that survive, neither of which the model represents. We report the two quantities separately and claim no proportionality between them, which is also why the abstract quotes a traffic ratio and a latency ratio as separate numbers.

6 Related work

Large receptive fields in CNNs. The effective receptive field of a deep CNN grows only as $\mathcal{O}(\sqrt{n})$ in depth (Luo et al., 2016), motivating dilation (Yu & Koltun, 2016) and directly enlarged kernels (Ding et al., 2022; Liu et al., 2023), which pay in parameters and FLOPs. WTConv (Finder et al., 2024) instead makes parameters logarithmic in the receptive field. This literature optimises parameters and FLOPs; our point is that for WTConv neither quantity is what determines runtime.

Wavelets in deep learning. Multiresolution analysis and the filter-bank algorithm (Mallat, 1989) on the Haar basis (Haar, 1910) underpin a line of work using the DWT as an aliasing-free replacement for pooling and downsampling (Williams & Li, 2018; Li et al., 2020) and as a multi-scale backbone for restoration (Liu et al., 2018). All of these invoke the transform through a framework `conv2d`, treating it as a fixed primitive. We treat it instead as a fusion boundary to be dissolved.

Fast wavelet transforms. The lifting scheme factors a wavelet transform into in-place prediction and update steps (Sweldens, 1996; 1998; Daubechies & Sweldens, 1998), reducing operation count and enabling in-place evaluation; GPU implementations have long compared filter-bank against lifting formulations (Tenllado et al., 2008). That literature optimises the transform in isolation. The transform is not the bottleneck here: our gain comes from fusing it into the surrounding convolution so that it never touches memory at all, and for Haar specifically the lifting factorisation coincides with the butterfly of Equation (5).

Kernel fusion and memory-bound operators. That data movement rather than arithmetic limits deep learning is well established (Ivanov et al., 2021; Wahib & Maruyama, 2014), and FlashAttention (Dao et al., 2022; Dao, 2024) is the canonical demonstration that an exact reassociation which keeps intermediates on chip can transform an operator's practical cost. Our work is the same argument applied to a different operator, with the additional structure that the transform being fused is multiplication-free, symmetric, and self-inverse. These properties make recomputation inexpensive and let analysis and synthesis exchange roles during backpropagation. General-purpose compilers (Ragan-Kelley et al., 2013; Chen et al., 2018; Sabne, 2020; Ansel et al., 2024) automate fusion of elementwise and reduction chains, but the reassociations of Section 4.2, collapsing an L-step recursion into a closed-form bit-indexed sum, rest on properties of the Haar basis that a general compiler does not have access to.

Depthwise convolution efficiency. Depthwise convolutions have low arithmetic intensity and are known to underutilise GPUs relative to their FLOP count (Howard et al., 2017; Ma et al., 2018; Lu et al., 2022). WTConv inherits this and compounds it with a per-level chain of full-tensor intermediates.

7 Limitations

Scope of the kernels. They implement Haar (`db1`) only. The reference layer accepts any PyWavelets family, and a model trained with `db2` or longer cannot use our kernels. The multiplication-free argument of Section 4.1 is specific to Haar; longer filters would still admit the reformulation but with a materially worse recompute trade. We also require depthwise operation with $C_{\text{in}} = C_{\text{out}}$, odd k, and $L \leq 5$ (the last is a hand-unrolled dispatch limit, not an algorithmic one).

Compilation. All results are eager-mode on both arms. Our layer opts out of Dynamo tracing, so we cannot report a fair `torch.compile` comparison; an inductor-generated fusion of the reference is a baseline this paper does not measure, and a reviewer is right to want it. We expect inductor to fuse the elementwise scale but not to discover Proposition 2.

Measurement. Single-layer microbenchmarks on one GPU per platform, with a fixed resident input tensor and no host-to-device transfer. We report geometric means over the configuration sweep and per-iteration dispersion, but no cross-machine replication. All reported ratios are layer-level, and Amdahl's law applies: a 5× layer speedup yields a smaller whole-model speedup, and we do not report an end-to-end training measurement here.

Shift-variance. As is already true of the reference operator, the 2^ℓ-aligned decomposition grid makes WTConv equivariant only to translations by multiples of 2^L. Our reformulation is exactly equivalent, so it neither introduces nor removes this property.

8 Conclusion

WTConv's difficulty was never its arithmetic. Expressed as a chain of framework primitives it runs at under 2% of a modern GPU's arithmetic capability, spending its time moving tensors that need never have existed. Accounting for those bytes exactly, and removing them with three algebraic identities (a register-resident multiplication-free analysis, a synthesis cascade collapsed into a bit-indexed closed form, and a scale folded into the weights), yields a bit-exact reformulation whose cost is within a small constant of the compulsory traffic. We do not claim that the ratio of bytes predicts the ratio of seconds; what the account buys is the identification of the quantity to remove. The broader point is that an operator's asymptotic parameter or FLOP advantage says little about whether it is usable; for the growing class of layers built from cheap, structured, multi-stage transforms, the I/O cost model is the one that predicts practice.

Broader Impact Statement

This work reduces the time and energy required to evaluate an existing operator without changing its function; the primary effect is lower computational cost for practitioners already using WTConv. We see no application-specific ethical concerns beyond those already attached to efficient computer vision generally.

References

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 929–947, 2024. doi: 10.1145/3620665.3640366.

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 578–594, 2018.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pp. 16344–16359, 2022.

Ingrid Daubechies and Wim Sweldens. Factoring wavelet transforms into lifting steps. *Journal of Fourier Analysis and Applications*, 4(3):247–269, 1998. doi: 10.1007/BF02476026.

Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding. Scaling up your kernels to 31x31: Revisiting large kernel design in CNNs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11963–11975, 2022.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

Shahaf E. Finder, Roy Amoyal, Eran Treister, and Oren Freifeld. Wavelet convolutions for large receptive fields. In Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (eds.), *Computer Vision – ECCV 2024*, volume 15112 of *Lecture Notes in Computer Science*, pp. 363–380, Cham, 2024. Springer. doi: 10.1007/978-3-031-72949-2_21.

Alfred Haar. Zur theorie der orthogonalen funktionensysteme. *Mathematische Annalen*, 69(3):331–371, 1910. doi: 10.1007/BF01456326.

Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, pp. 711–732, 2021.

Qiufu Li, Linlin Shen, Sheng Guo, and Zhihui Lai. Wavelet integrated CNNs for noise-robust image classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7243–7254, 2020.

Pengju Liu, Hongzhi Zhang, Kai Zhang, Liang Lin, and Wangmeng Zuo. Multi-level wavelet-CNN for image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pp. 773–782, 2018.

Shiwei Liu, Tianlong Chen, Xiaohan Chen, Xuxi Chen, Qiao Xiao, Boqian Wu, Tommi Kärkkäinen, Mykola Pechenizkiy, Decebal Constantin Mocanu, and Zhangyang Wang. More ConvNets in the 2020s: Scaling up kernels beyond 51x51 using sparsity. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 10012–10022, 2021.

Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11976–11986, 2022.

Gangzhao Lu, Weizhe Zhang, and Zheng Wang. Optimizing depthwise separable convolution operations on GPUs. *IEEE Transactions on Parallel and Distributed Systems*, 33(1):70–87, 2022. doi: 10.1109/TPDS.2021.3084813.

Wenjie Luo, Yujia Li, Raquel Urtasun, and Richard Zemel. Understanding the effective receptive field in deep convolutional neural networks. In *Advances in Neural Information Processing Systems 29 (NIPS 2016)*, pp. 4898–4906, 2016.

Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. ShuffleNet V2: Practical guidelines for efficient CNN architecture design. In *Computer Vision – ECCV 2018*, volume 11218 of *Lecture Notes in Computer Science*, pp. 122–138. Springer, 2018. doi: 10.1007/978-3-030-01264-9_8.

Stéphane G. Mallat. A theory for multiresolution signal decomposition: The wavelet representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(7):674–693, 1989. doi: 10.1109/34.192463.

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 519–530, 2013. doi: 10.1145/2491956.2462176.

Amit Sabne. XLA: Compiling machine learning for peak performance. Google Research, 2020. URL `https://research.google/pubs/xla-compiling-machine-learning-for-peak-performance/`.

Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

Wim Sweldens. The lifting scheme: A custom-design construction of biorthogonal wavelets. *Applied and Computational Harmonic Analysis*, 3(2):186–200, 1996. doi: 10.1006/acha.1996.0015.

Wim Sweldens. The lifting scheme: A construction of second generation wavelets. *SIAM Journal on Mathematical Analysis*, 29(2):511–546, 1998. doi: 10.1137/S0036141095289051.

Christian Tenllado, Javier Setoain, Manuel Prieto, Luis Piñuel, and Francisco Tirado. Parallel implementation of the 2D discrete wavelet transform on graphics processing units: Filter bank versus lifting. *IEEE Transactions on Parallel and Distributed Systems*, 19(3):299–310, 2008. doi: 10.1109/TPDS.2007.70716.

Mohamed Wahib and Naoya Maruyama. Scalable kernel fusion for memory-bound GPU applications. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 191–202, 2014. doi: 10.1109/SC.2014.21.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Travis Williams and Robert Li. Wavelet pooling for convolutional neural networks. In *International Conference on Learning Representations (ICLR)*, 2018. URL `https://openreview.net/forum?id=rkhlb8lCZ`.

Fisher Yu and Vladlen Koltun. Multi-scale context aggregation by dilated convolutions. In *International Conference on Learning Representations (ICLR)*, 2016.

A Reproducibility

Every number in this paper is produced by a script in the accompanying artifact, and no value is transcribed into the manuscript by hand: the benchmark scripts emit JSON, a table generator turns that JSON into the LaTeX fragments the paper `\input`s, and re-running the pipeline overwrites the tables in place.

A.1 Pipeline

```
python scripts/bench.py         --device cuda --protocol both --mode both \
                                --methods depthwise,dense,reference,fused_cuda \
                                --out results/bench_cuda.json
python scripts/correctness.py   --device cuda \
                                --out results/correctness.json
python scripts/make_tables.py   --bench results/bench_cuda.json \
                                --correctness results/correctness.json
```

`scripts/common.py` holds the measurement harness and the weight synchronisation used by both entry points; `scripts/bench.py` sweeps the configuration grid; `scripts/correctness.py` performs the equivalence tests; `scripts/make_tables.py` renders the tables.

A.2 Measurement protocol

Timing. CUDA events bracket each individual iteration, so device time is measured rather than host-side launch time, and the distribution over iterations is retained rather than only its mean. Each configuration runs 20 warmup iterations — enough to cover JIT compilation and cuDNN algorithm selection — followed by 50 timed iterations. We report the geometric mean of per-configuration ratios over the sweep; the raw JSON also carries median, standard deviation and the 10th/90th percentiles for every configuration, which we recommend inspecting before quoting any single figure.

Memory. `torch.cuda.reset_peak_memory_stats` followed by one untimed step and `torch.cuda.max_memory_allocated`. This measures allocator high-water mark, not reserved memory.

Weight synchronisation. Correctness comparisons construct a reference module and a fused module from the same seed and then copy every learnable tensor from the former into the latter, so the two modules are the same function. The base path maps `base_conv.weight`/`.bias`/`base_scale.weight` onto the fused module's corresponding parameters, and level ℓ maps `wavelet_convs[l].weight` and `wavelet_scale[l].weight`. The subband axis is packed identically in both (channel $= 4c + s$ with $s \in (\mathrm{LL}, \mathrm{LH}, \mathrm{HL}, \mathrm{HH})$); rather than assume this, `correctness.py` drives the reference's own filter bank with a one-hot 2×2 patch and checks the resulting coefficient order and signs against Eq. (5).

Cotangent. Backward comparisons use a fixed random cotangent rather than `.sum()`, which would leave sign-cancelling errors undetected.

Tolerances. `fp32` is held to atol $= 10^{-5}$, rtol $= 10^{-4}$; `fp16` to $2\times10^{-2}/10^{-2}$; `bf16` to $10^{-1}/5\times10^{-2}$. These follow from the accumulated rounding of a k^2-term dot product in the respective format rather than being tuned to make the test pass. We note that an absolute tolerance of 10^{-7}, sometimes quoted for comparisons of this kind, is not attainable for a multi-level accumulation in `fp32` and should not be expected.

A.3 Configuration grid

$C \in \{32, 64, 128\}$; $H = W \in \{128, 256, 512\}$; batch size 8; $L \in \{1, \ldots, 5\}$; $k = 3$ for WTConv and, under the homogeneous protocol, for the plain-convolution baselines; $k = 7$ for the plain-convolution baselines under the receptive-field-matched protocol; `fp32` and `fp16` for the latency and memory sweep, with `bf16` additionally covered by the equivalence tests. Each of the two protocols is measured both for a forward pass and for a forward-and-backward step, giving the 864 measurements reported in Section 5. Baselines: *depthwise* is `nn.Conv2d(C, C, k, groups=C, padding='same')` and *dense* is `nn.Conv2d(C, C, k, padding='same')`.

A.4 Environment

The environment record (GPU, compute capability, HBM capacity, PyTorch, CUDA, and cuDNN versions) is captured automatically into every results JSON and rendered into the manuscript by the table generator, so the reported hardware cannot drift from the hardware actually used.

A.5 Claim-to-evidence map

Claim	Evidence
Haar analysis is self-inverse and self-adjoint	Proposition 1; verified exactly over the rationals by `correctness.py`
Single-pass synthesis	Proposition 2
Reference moves $7N + \frac{43}{3}N(1 - 4^{-L})$ elements	Per-primitive accounting, Section 3, read off the reference implementation stage by stage
Operator is bandwidth-bound	$I \in [0.44, 0.89]$ FLOP/byte against $I^\star \approx 50$, Section 3
Fusion is exact, not approximate	Each step is an algebraic identity; residuals reported by `correctness.py` in Table 7 against the tolerances above
Fused form moves $7N$ (L=1) to $8.27N$ elements	Per-stage accounting, Section 4.4; not converted into a predicted latency ratio
Measured speedup	Tables 3 and 4, against the reference and against plain convolutions at matched kernel size and matched receptive field
Measured peak memory	Table 5 (training step) and Table 6 (inference)
Measured speedup exceeds the traffic ratio	Section 5.4: launch count and occupancy, not additional traffic removed

A.6 Known gaps

For transparency we list what the artifact does *not* establish. There is no accuracy result connecting the fused kernels to a downstream task, and this paper reports no end-to-end training measurement: every reported ratio is layer-level. There is no `torch.compile` arm on either side. Timing uses a single resident input tensor, so no host-to-device transfer or data loading is included. Each platform is represented by one device.

B Supplementary measurements

Both baseline protocols of Section 5.1, matched kernel size and matched receptive field, are already reported in the main text; this appendix carries the two tables that did not fit there.

Table 6: Peak allocated memory for inference (forward only, no autograd tape) relative to the reference WTConv (1.00×; higher means less memory), same sweep as Table 3.

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=3	`fp32`	2.46×	3.62×	3.53×	3.51×	3.51×
	`fp16`	1.55×	2.28×	2.22×	2.21×	2.20×
Dense conv, k=3	`fp32`	2.38×	3.50×	3.42×	3.40×	3.39×
	`fp16`	1.56×	2.29×	2.24×	2.23×	2.22×
Depthwise conv, k=7	`fp32`	1.80×	2.64×	2.58×	2.56×	2.56×
	`fp16`	1.55×	2.28×	2.22×	2.21×	2.20×
Dense conv, k=7	`fp32`	1.71×	2.51×	2.45×	2.43×	2.43×
	`fp16`	1.55×	2.28×	2.23×	2.21×	2.21×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	1.56×	2.04×	1.94×	1.91×	1.91×
	`fp16`	1.56×	2.04×	1.94×	1.91×	1.91×

Table 7: Numerical agreement with the reference implementation. Entries are maximum absolute deviation over the whole tensor, at L decomposition levels, for the forward output and for every gradient the layer produces. The fused kernels compute the same multilinear map with a different association order, so the residual is floating-point reassociation error alone.

Backend	dtype	L	forward	∇_X	∇_W	∇_s
Fused (CUDA)	`fp32`	1	2.4e-07	2.4e-07	4.8e-06	1.7e-05
Fused (CUDA)	`fp32`	2	2.4e-07	2.4e-07	3.3e-06	1.9e-05
Fused (CUDA)	`fp32`	3	2.4e-07	2.4e-07	1.9e-06	1.1e-05
Fused (CUDA)	`fp32`	4	2.4e-07	2.4e-07	1.1e-06	3.8e-06
Fused (CUDA)	`fp32`	5	2.4e-07	2.4e-07	9.5e-07	4.8e-06
Fused (CUDA)	`fp16`	1	2.0e-03	2.0e-03	7.8e-03	6.2e-02
Fused (CUDA)	`fp16`	2	2.0e-03	2.0e-03	3.9e-03	3.1e-02
Fused (CUDA)	`fp16`	3	2.0e-03	2.0e-03	3.9e-03	1.6e-02
Fused (CUDA)	`fp16`	4	2.0e-03	2.0e-03	9.8e-04	3.9e-03
Fused (CUDA)	`fp16`	5	2.0e-03	2.0e-03	9.8e-04	2.0e-03
Fused (CUDA)	`bf16`	1	1.6e-02	1.6e-02	6.2e-02	3.1e-01
Fused (CUDA)	`bf16`	2	1.6e-02	1.6e-02	3.1e-02	2.5e-01
Fused (CUDA)	`bf16`	3	1.6e-02	1.6e-02	1.6e-02	6.2e-02
Fused (CUDA)	`bf16`	4	1.6e-02	1.6e-02	7.8e-03	3.1e-02
Fused (CUDA)	`bf16`	5	1.6e-02	1.6e-02	3.9e-03	3.1e-02

Table 6 is the inference counterpart of Table 5: peak allocated memory for a forward pass with no autograd tape. The reduction over the reference is smaller than in training (1.56–2.04× against 1.85–2.34×), which is the expected direction, since a large part of what fusion removes is tape-retained subband tensors that inference never allocates in the first place.

Table 7 quantifies the numerical agreement summarised in Section 5.1. The deviations are those of a k^2-term accumulation in the working precision and do not grow with L: the forward output and ∇_X sit at one unit in the last place of the working format at every level, and the parameter gradients *shrink* with depth because the level-ℓ weight gradient is a reduction over $4^{-(\ell-1)}$ as many terms.